

Custody

Proving digital evidence has not been changed

Technical write-up. Team Captain. ICSC 2026 Universities Hackathon, Track H.

1. The problem, and what we built

Cybercrime cases, fraud investigations and disciplinary panels increasingly turn on digital evidence. That evidence is only useful if it can be shown to be genuine and unchanged since collection. In practice it is copied onto flash drives, emailed between officers, saved on shared machines and handed over as a folder. Whether the copy matches the original, who touched it, and whether anything changed are unanswerable. Cases are lost on integrity grounds, not because anyone tampered with the evidence but because nobody can prove they did not.

Custody is a working chain of custody system. Evidence is fingerprinted at the moment of collection, on the collecting officer's own device, before the file goes anywhere else. Every hand it passes through is recorded in a tamper evident log. Any later change is provable and localisable to a byte range. The output is a one page document a panel member with no technical background can read and act on. It runs on one laptop, uses only free and open source software, and depends on no network.

There are three surfaces: a React Native field app for offline collection, an Express and PostgreSQL backend holding the chain, and a Next.js dashboard for case officers. All of it is built and running.

2. How integrity is proven

Two questions are answered separately throughout, because they fail for different reasons and carry different consequences: has the file changed, and has the handling record itself been edited? A file can be intact while its custody trail has been tampered with, and the reverse. Collapsing them into a single indicator would conceal exactly the distinction a panel needs.

The fingerprint	SHA-256 over the file's contents, taken on the collecting device before the file is handled by anyone else, and recomputed on demand. Nothing between the scene and the server can alter the file undetectably.
The item chain	Every custody event stores the hash of the event before it. Altering an event invalidates every hash after it, and the first failure locates the change to an exact entry.
The case chain	One event per item creation. Its reference to the item is deliberately not a foreign key, so the record survives the item's deletion and the gap becomes visible. Without this, deleting an entire item would leave no trace.

Verification never trusts a stored hash. It recomputes every event hash from that event's own fields and checks two conditions at each step: that the recorded hash matches what the contents produce, and that the backward link matches the previous event's hash. The first failure is reported with its position, which is what lets the dashboard render a visibly severed link at the exact point the record was edited, with everything above it still verified.

Custody events are append only, enforced by a database trigger rather than by application code, because a guard written only in the application is one a direct database connection walks past. A mistake is corrected by appending a correction event that references the erroneous one. Nothing is ever edited or deleted, which mirrors how physical evidence handling already works.

3. Large files: chunked hashing and a Merkle tree

A single phone extraction can exceed 100GB, so files are never hashed as one blob. A file is read in 4MB chunks, each chunk is hashed, and a binary Merkle tree is built over the ordered chunk hashes, with an unpaired node promoted unchanged to the next level. The root is the item fingerprint; the chunk list is stored beside it. Three consequences follow, and each is measured rather than asserted.

Memory use stays flat as the file grows, because the whole file is never held at once. Measured heap growth while hashing the 30MB test file was under one megabyte.

A change is located, not merely detected. Altering a single byte at offset 21,000,000 of the 30MB file reports chunk 6 of 8 and the range 20,971,520 to 25,165,823. In a hearing, naming the affected part of a file is a materially different claim from reporting a mismatch.

Verification can be paused and resumed after a power cut, because chunk hashes are independent of one another.

We tested at 30MB and extrapolated. We did not test a 100GB extraction. What is defensible is that memory use is constant regardless of file size, and that is measured, not claimed.

4. Collection when there is no network

The requirement to work through power and network cuts is answered by the architecture rather than by a policy. At a scene the officer taps once, the file is fingerprinted on the device, and the record is written to local storage. No network call is anywhere on that path. Sync is a separate, deferred, interruptible activity, and the evidence file itself is never uploaded: only the fingerprint and metadata travel, which avoids request size limits entirely and mirrors evidence handling, where the exhibit and the paperwork move separately.

Offline events carry the device clock, which a defence could challenge. We do not hide this. The server stamps its own time on arrival and the report states both in words, naming the gap: recorded on the device at 14:32, confirmed by the server four hours and thirty five minutes later, once the connection was restored. This is a deliberate design decision, not an oversight.

The same honesty produced a distinction we did not originally have. A device seals an item at the scene and can know nothing about the evidence store's layout, so a freshly synced item is reported as awaiting its exhibit rather than as missing. Reporting it as missing would assert that evidence had been lost, when in fact the paperwork had simply arrived before the exhibit.

5. The report

The brief's final audience is a magistrate or panel member who is not a technologist, so the report is one page, in Times, with no hashes and no jargon on it. Technical detail is confined to an appendix that can be ignored. Two things are enforced in code: no database enum reaches the page, and no ISO timestamp does, because those are the two things that identify a document as machine output. The dashboard is checked for the same leaks by an automated test.

Is it unchanged?	Yes. The item is identical to what was collected.
How do we know?	A digital fingerprint was taken at the moment of collection...
Who has handled it?	14 Apr 2026, 11:20 Mr T. Okonjo, collected from the handset
Can this list be edited?	No. Each entry is locked to the one before it.

6. Design decisions worth defending

No blockchain	A blockchain solves distrust between parties with no shared authority. A court or a university panel has one. A hash chain, append only storage and periodically published root hashes give the same tamper evidence at a fraction of the complexity, and can be explained to a panel member in a single sentence.
Local only	No cloud dependency and no paid service. The system is demonstrated against a local database, because demonstrating offline first architecture against a hosted one would be a contradiction.
Server authority	The server stamps its own time and recomputes every hash rather than accepting the client's. A device cannot know the server's identifiers before an item exists there, so the field app sends no event hashes at all. The handset claims; the server records.

7. What we tested

240 assertions across six suites, all passing. The suites are not only route checks: they assert the exact field names each consumer reads, so a renamed field fails in testing rather than in front of an audience.

Device and server parity	13 assertions. The field app's hashing transliterated into Node and compared against the server's on real files, including tree shape at 1, 2, 3, 5, 7, 8 and 9 chunks. If these two implementations ever diverge, every fingerprint taken in the field becomes unverifiable, so this is the suite that matters most.
API and contracts	104 assertions. Every endpoint, response shapes, error paths, malformed input and injection attempts in path parameters.
Offline sync	47 assertions across two suites, one of which imports the field app's own payload builder rather than a copy of it and posts its output at the live API.
Dashboard	53 assertions against the served HTML: no raw enum, ISO timestamp or undefined reaching a reader, and the broken chain rendering at the correct link.
Acceptance	23 assertions. The full demonstration end to end, including the deliberate tampering and the reset, run before every rehearsal.

8. Where the data came from

Everything in this system is synthetic and was generated by the team. There are no real persons, cases, accounts, student records, transactions or personal data of any kind. Names, badge numbers and account numbers are invented. The stand-in for a phone extraction is 30MB of random bytes; it is not a real extraction and does not need to be, because the system fingerprints bytes and never interprets contents. The provenance is documented in the repository README and in the test data pack.

9. Honest limits

All of these are true, and we would rather state them than be asked.

Corrupt collector	If the officer alters the file before sealing it, the system faithfully proves the integrity of already altered evidence. We protect the chain from collection onward, not before it. This is real and unsolved.
Device clock	Offline timestamps come from the device and can be wrong or deliberately set. We show both clocks rather than hiding the gap, but the device time is not independently trustworthy.
Scale	Tested at 30MB and extrapolated. A full extraction was never hashed.
Per officer signing	Events are attributable to a login, not cryptographically to a person. Signing each event with an officer held key is the correct upgrade. We describe it; we did not build it.
Storage	We prove change. We do not prevent deletion of the underlying file.
Bulk compromise	An attacker with full database and server access could rebuild the entire chain consistently. Publishing periodic root hashes to an external location is the mitigation. We describe it; we did not build it.

10. Running it

```
brew services start postgresql@16      # or: sudo service postgresql start
cd hackathonBackend && npm install && npm run demo:reset && npm run dev      # API   :4000
cd hackathonWebApp   && npm install && npm run dev                          # web   :3000
npm test && npm run test:web && npm run test:acceptance                      # 240 assertions
```

Full setup, the demonstration script and the testing guide are in the repository as README.md, DEMO.md and TESTING.md.